

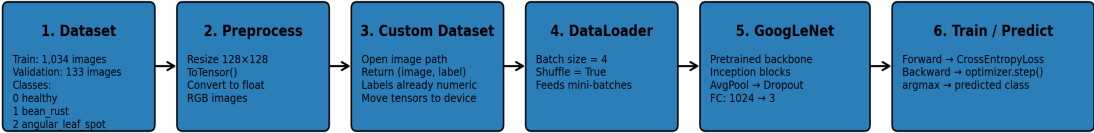
Bean Leaf Lesion Classification with Pretrained GoogLeNet

A detailed explanation of pretrained_model_DL04.ipynb

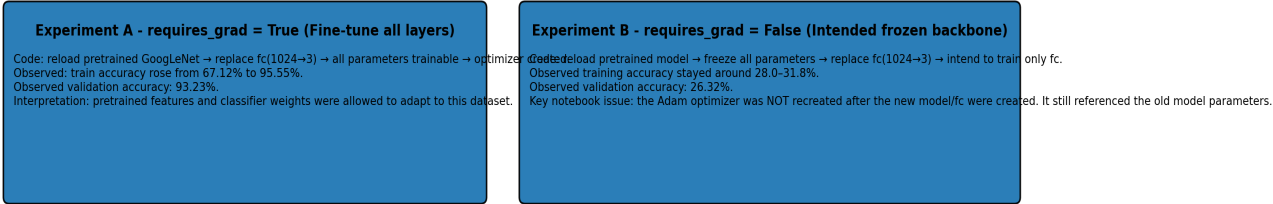
This guide follows the notebook as written and focuses especially on the two experiments using `requires_grad=True` and `requires_grad=False`.

Bean Leaf Lesion Classification - Pretrained GoogLeNet Workflow

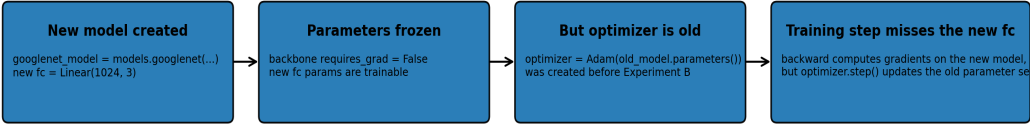
Source: pretrained_model_DL04.ipynb | 3 classes | PyTorch | 15 epochs | batch size 4



Two training experiments in the notebook



Why the second experiment collapsed



Recorded headline results: fine-tuning run reached 95.55% training accuracy and 93.23% validation accuracy. The later frozen-backbone run stayed around 28-32% training accuracy and reached 26.32% validation accuracy.

1. What the notebook is doing

The notebook builds a three-class bean leaf image classifier. It downloads the dataset, loads train/validation CSV files, turns file names into absolute paths, prepares images with torchvision transforms, wraps the data in a custom PyTorch Dataset, batches it with DataLoader, and trains a pretrained GoogLeNet model.

Dataset recorded in the notebook

Split	Rows
Train	1,034
Validation	133

Classes

The CSVs contain numeric categories 0, 1, and 2. The folder names shown by the notebook are **healthy**, **bean_rust**, and **angular_leaf_spot**. Training counts are 341, 345, and 348 respectively.

Preprocessing

```
transform = transforms.Compose([
    transforms.Resize((128, 128)),
    transforms.ToTensor(),
    transforms.ConvertImageDtype(torch.float)
])
```

Images are opened with PIL, converted to RGB, resized to 128x128, converted to tensors, and converted to floating-point tensors.

2. Custom Dataset and DataLoader

CustomImageDataset is the bridge between the DataFrame and PyTorch. Its three important methods have simple jobs:

__init__ stores the DataFrame, transform, and numeric labels. **__len__** returns the number of rows.

__getitem__(idx) finds one image, opens it, applies transforms, and returns **(image, label)**.

```
class CustomImageDataset(Dataset):
    def __len__(self):
        return self.dataframe.shape[0]

    def __getitem__(self, idx):
        img_path = self.dataframe.iloc[idx, 0]
        label = self.labels[idx]
        image = Image.open(img_path).convert("RGB")
        if self.transform:
            image = self.transform(image).to(device)
        return image, label
```

The DataLoader then groups these individual samples into mini-batches. The notebook uses **BATCH_SIZE = 4**, so the model sees four images at a time.

Conceptually:

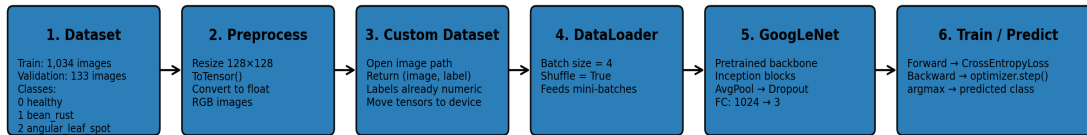
folders/CSV → Dataset → DataLoader → batch of images + labels → model

3. Pretrained GoogLeNet

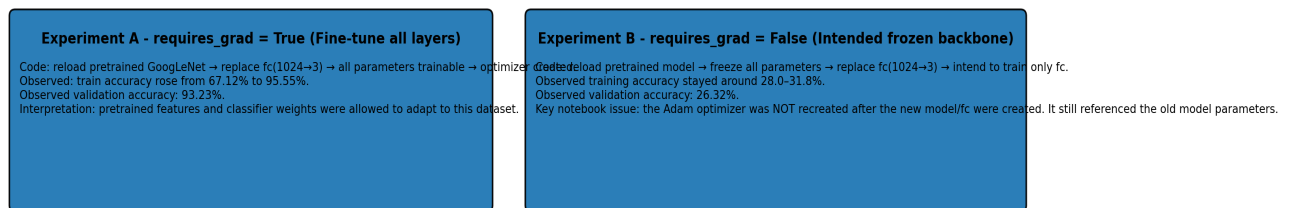
The notebook creates a pretrained GoogLeNet model with `models.googlenet(weights='DEFAULT')`. It then replaces the final classifier because the original classifier is not configured for this notebook's three target classes.

Bean Leaf Lesion Classification - Pretrained GoogLeNet Workflow

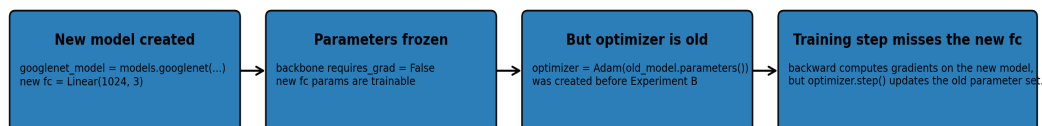
Source: pretrained_model_DL04.ipynb | 3 classes | PyTorch | 15 epochs | batch size 4



Two training experiments in the notebook



Why the second experiment collapsed



The final classifier shown in the notebook is:

```
Linear(in_features=1024, out_features=3, bias=True)
```

So the final stage maps 1024 extracted features to three class scores. The prediction is obtained with **argmax** across the class dimension.

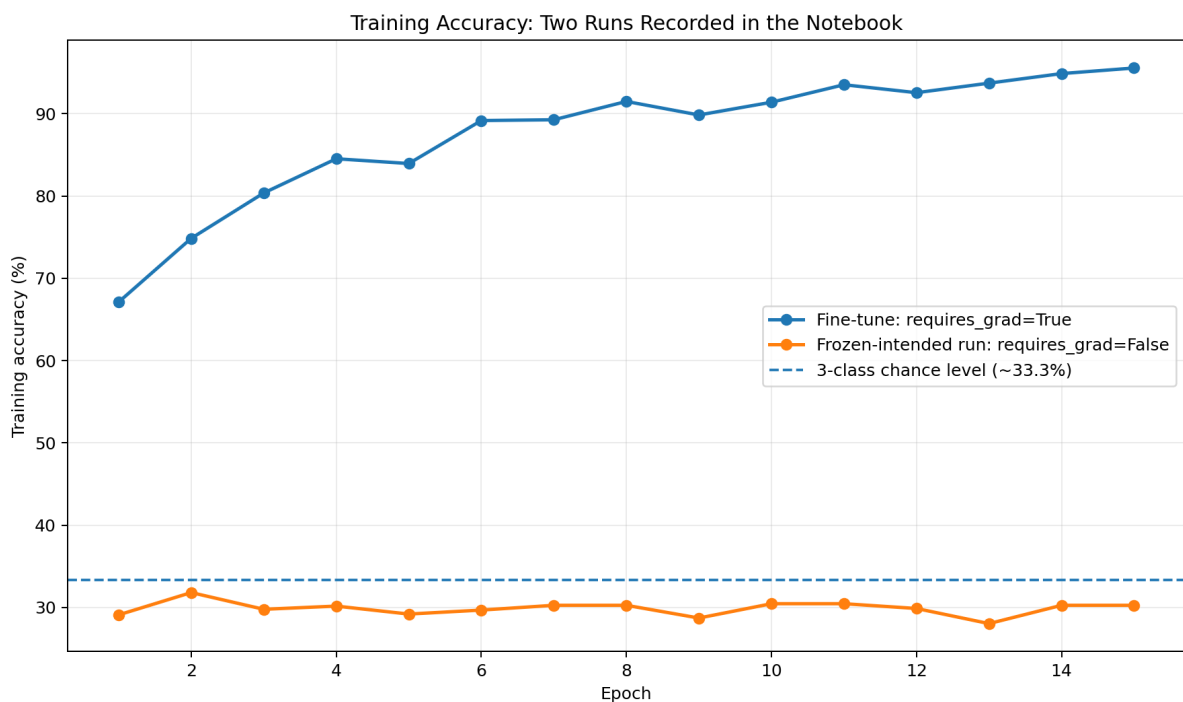
4. Experiment A: requires_grad=True

The first experiment makes all model parameters trainable:

```
for param in googlenet_model.parameters():  
    param.requires_grad = True  
  
googlenet_model.fc = torch.nn.Linear(  
    googlenet_model.fc.in_features, num_classes  
)  
  
optimizer = Adam(googlenet_model.parameters(), lr=LR)
```

This is a **fine-tuning** setup. The pretrained backbone is not frozen, so gradients can flow through the entire network and the optimizer can update the full parameter set.

Recorded training accuracy



Final recorded training accuracy: **95.5513%**. Recorded validation accuracy: **93.23%**.

5. Experiment B: requires_grad=False

The second experiment intends to freeze the pretrained network and train only the newly replaced final classifier:

```
googlenet_model = models.googlenet(weights="DEFAULT")

for param in googlenet_model.parameters():
    param.requires_grad = False

googlenet_model.fc = torch.nn.Linear(
    googlenet_model.fc.in_features, num_classes
)
googlenet_model.fc.requires_grad = True
```

The intended idea is correct: keep pretrained feature extraction fixed and learn a small task-specific classifier. However, the notebook has a critical implementation problem immediately before this experiment.

6. Why the second run performed so differently

Main issue: the optimizer was not recreated.

The notebook created Adam during the first experiment:

```
optimizer = Adam(googlenet_model.parameters(), lr=LR)
```

Then the notebook created a completely new GoogLeNet instance for the frozen experiment and replaced its classifier. But it did not create a new Adam optimizer after that replacement.

```
# Correct frozen-backbone transfer learning
googlenet_model = models.googlenet(weights="DEFAULT")

# Freeze pretrained backbone
for param in googlenet_model.parameters():
    param.requires_grad = False

# Replace classifier for 3 classes
googlenet_model.fc = nn.Linear(
    googlenet_model.fc.in_features, num_classes
)

# New fc parameters are trainable by default
googlenet_model.to(device)

# IMPORTANT: create a NEW optimizer for the NEW model
optimizer = Adam(
    filter(lambda p: p.requires_grad, googlenet_model.parameters()),
    lr=LR
)
```

Because the optimizer holds references to the parameter objects it was given, the old optimizer is not automatically redirected to the newly created model and newly created fc layer. The second run therefore has the wrong optimizer/model pairing.

Recorded result: training accuracy stayed around 28.0-31.8%, and validation accuracy was 26.32%. With three classes, 33.3% is the simple chance reference. The result is therefore consistent with a model that is not successfully learning. The notebook does not show a correctly implemented frozen-backbone baseline.

7. Correct frozen-backbone transfer learning

The important fix is to create the optimizer **after** freezing the backbone and creating the new classifier, and to pass only trainable parameters to the optimizer.

```
googlenet_model = models.googlenet(weights="DEFAULT")

for param in googlenet_model.parameters():
    param.requires_grad = False

googlenet_model.fc = nn.Linear(
    googlenet_model.fc.in_features, num_classes
)

googlenet_model.to(device)

optimizer = Adam(
    filter(lambda p: p.requires_grad,
           googlenet_model.parameters()),
    lr=LR
)
```

In this corrected setup, the pretrained layers remain fixed while the newly created classifier is trainable and is explicitly included in the new optimizer.

Important distinction

The notebook's 95.55% vs 30% difference should not be interpreted as 'True is always better than False'. The recorded comparison is confounded by the optimizer bug in the second run.

8. Additional notebook notes

Printed loss scaling: total training loss is divided by 1000 before being plotted/printed. This is not a standard average over the actual number of batches.

Validation mode: the notebook uses `torch.no_grad()`, but it does not explicitly call `model.eval()` before the validation pass. Since GoogLeNet contains dropout and batch-normalization layers, an evaluation-mode pass is normally used for deterministic validation behavior.

Validation shuffle: `val_loader` uses `shuffle=True`. For evaluation, `shuffle=False` is normally clearer and makes sample order deterministic.

Device movement: images are moved to the device inside `__getitem__`. This works in the recorded notebook, although it is common to move batches to the device inside the training loop instead.

Final mental model

Pretrained model → choose which parameters can learn → replace the classifier → create the correct optimizer → train → evaluate.

The most important lesson from this notebook is that **requires_grad controls gradient computation, but the optimizer must also contain the parameters you want to update.**